

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Научный руководитель, доцент
департамента программной
инженерии ФКН, кандидат физико-
математических наук

_____ Г. Н. Жукова
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
«Программная инженерия», старший
преподаватель департамента
программной инженерии

_____ Н. А. Павлочев
«__» _____ 2026 г.

**ПРИЛОЖЕНИЕ ДЛЯ ОБРАБОТКИ ТЕКСТА, ПОЛУЧЕННОГО NBSCONVERT
И СОДЕРЖАЩЕГО ФОРМУЛЫ LATEX**

Текст программы

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.06.11-01 12 01-1-ЛУ

Исполнитель:

Студент группы БПИ247

_____ / А. А. Малапура /

«__» _____ 2026 г.

Инва.№ подп	Подп. и дата
Взам. инв.№	Инва.№ дубл.
Подп. и дата	Подп. и дата

УТВЕРЖДЕН

RU.17701729.06.11-01 12 01-1-ЛЮ

**ПРИЛОЖЕНИЕ ДЛЯ ОБРАБОТКИ ТЕКСТА, ПОЛУЧЕННОГО NB CONVERT
И СОДЕРЖАЩЕГО ФОРМУЛЫ LATEX**

Текст программы

RU.17701729.06.11-01 12 01-1

Листов 51

Инв.№ подп	Подп. и дата	Взам. инв.№	Инв.№ дубл.	Подп. и дата

АННОТАЦИЯ

В настоящем документе «Текст программы» приведены исходные тексты программных модулей программного средства FormuLab.

Программа разработана на языке программирования Python с использованием библиотеки Tkinter.

Среда разработки — Visual Studio Code.

FormuLab представляет собой desktop-приложение для преобразования файлов Jupyter Notebook (.ipynb) в формат LaTeX (.tex). Программа реализована на языке программирования Python с использованием библиотеки Tkinter для построения графического пользовательского интерфейса.

В документ включены исходные тексты модулей, обеспечивающих запуск приложения, навигацию между окнами, загрузку notebook-файлов, выбор ячеек, конвертацию данных в LaTeX, обработку Markdown-заголовков, математических формул, изображений и пакетного режима сохранения.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
 2. ГОСТ 19.102-77 [2]: Стадии разработки.
 3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
 4. ГОСТ 19.104-78 [4]: Основные надписи.
 5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.
 6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
 7. ГОСТ 19.401-78 [10]: Текст программы. Требования к содержанию и оформлению.
- Изменения к данному документу оформляются согласно ГОСТ 19.603-78 [8], ГОСТ 19.604-78 [13].

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ТЕКСТ ПРОГРАММЫ	4
1.1. Основные файлы приложения	4
1.1.1. Листинг 1. Файл src/main.py	4
1.1.2. Листинг 2. Файл src/formulab_application.py	5
1.2. Модели	7
1.2.1. Листинг 3. Файл src/MVC/models/main_menu_model.py	7
1.2.2. Листинг 4. Файл src/MVC/models/cell_selector_model.py	9
1.2.3. Листинг 5. Файл src/MVC/models/file_finalization_model.py	17
1.3. Представления	34
1.3.1. Листинг 6. Файл src/MVC/views/main_menu_view.py	34
1.3.2. Листинг 7. Файл src/MVC/views/cell_selector_view.py	35
1.3.3. Листинг 8. Файл src/MVC/views/file_finalization_view.py	41
1.4. Контроллеры	42
1.4.1. Листинг 9. Файл src/MVC/controllers/main_menu_controller.py	43
1.4.2. Листинг 10. Файл src/MVC/controllers/cell_selector_controller.py	44
1.4.3. Листинг 11. Файл src/MVC/controllers/file_finalization_controller.py	46
1.5. Утилиты	47
1.5.1. Листинг 12. Файл src/utills/formulab_exceptions.py	47
1.5.2. Листинг 13. Файл src/utills/windows_subprocess.py	48
ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ	49

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ТЕКСТ ПРОГРАММЫ

Программа FormuLab реализована на языке программирования Python. Исходный код программы организован в соответствии с архитектурным шаблоном Model-View-Controller.

В состав исходного текста программы входят модули, расположенные в каталоге src/. Модули приложения разделены на основные файлы запуска, компоненты MVC-архитектуры и вспомогательные утилиты.

Проект развивается в отдельном репозитории автора на основе исходного прототипа [20]; история изменений отражена в коммитах.

Ознакомиться подробнее с текстом программы можно на сайте Github по ссылке:
<https://github.com/m4ls0n/FormuLabRefactor>

Ниже приведены исходные тексты программных модулей FormuLab.

1.1. Основные файлы приложения

К основным файлам приложения относятся модули, отвечающие за запуск программы и управление жизненным циклом графического интерфейса.

Файл src/main.py является точкой входа в приложение. В нем выполняется инициализация Tkinter и запуск основного класса приложения.

Файл src/formulab_application.py содержит класс FormuLabApplication, который управляет созданием окон приложения и переключением между основными экранами: главным меню, окном выбора ячеек и окном доработки итогового файла.

1.1.1. Листинг 1. Файл src/main.py

```
import os
import sys
import tkinter as tk
import warnings

from nbformat.warnings import MissingIDFieldWarning

from src.formulab_application import FormuLabApplication

def resource_path(rel_path):
    """Возвращает корректный путь к файлу как в режиме dev, так и в упакованном exe."""
    # Проверка на запуск release-версии.
    if hasattr(sys, "_MEIPASS"):
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

    # PyInstaller распаковал ресурсы сюда.
    # noinspection PyProtectedMember
    return os.path.join(sys._MEIPASS, rel_path)

# В режиме разработки поднимаемся на один уровень вверх (из папки src/ в корень
проекта).
base_path = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
return os.path.join(base_path, rel_path)

# Запуск приложения.
if __name__ == "__main__":
    import subprocess
    from src.utils.windows_subprocess import popen_hidden
    subprocess.Popen = popen_hidden

    warnings.filterwarnings("ignore", category=MissingIDFieldWarning)
    warnings.filterwarnings("ignore", message="IPython3 lexer unavailable, falling back
on Python 3")
    root = tk.Tk()
    # Устанавливаем иконку окна.
    icon_file = resource_path("assets\\logo.ico")
    # noinspection PyBroadException
    try:
        # для Windows.
        root.iconbitmap(icon_file)
    except Exception:
        # fallback (например, если iconbitmap не сработал).
        img = tk.PhotoImage(file=icon_file)
        root.iconphoto(False, img)
    app = FormuLabApplication(root)
    app.launch()

```

1.1.2. Листинг 2. Файл src/formulab_application.py

```

from src.MVC.controllers.main_menu_controller import MainMenuController
from src.MVC.controllers.cell_selector_controller import CellSelectorController
from src.MVC.controllers.file_finalization_controller import FileFinalizationController

class FormuLabApplication:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

def __init__(self, root):
    self.root = root

    # Текущий редактируемый файл Jupyter Notebook.
    self.editable_notebook_data = None
    self.notebook_sources = []

    width = self.root.winfo_screenwidth()
    height = self.root.winfo_screenheight()
    self.root.geometry("%dx%d" % (width, height))

    self.current_controller = MainMenuController(self)

def show_main_menu(self):
    # Переход к главному меню.
    if isinstance(self.current_controller, (CellSelectorController,
FileFinalizationController)):
        self.current_controller.view.pack_forget()
        self.current_controller = MainMenuController(self)
        self.current_controller.view.pack()

def show_cell_selector(self):
    # Переход к экрану выбора ячеек.
    if isinstance(self.current_controller, (MainMenuController,
FileFinalizationController)):
        self.current_controller.view.pack_forget()

    # Создаем контроллер для выбора ячеек с необходимыми данными.
    # Обновляем текущее состояние хранимого ноутбука в случае, если пользователь
выбрал новый файл.
    if isinstance(self.current_controller, MainMenuController):
        self.editable_notebook_data = self.current_controller.model.notebook_data
        self.notebook_sources = self.current_controller.model.notebook_sources
        self.current_controller = CellSelectorController(
            self,
            self.editable_notebook_data,
            self.notebook_sources
        )

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

self.current_controller.view.pack()

def show_file_finalization(self):
    # Переход к экрану доработки файла tex.
    if isinstance(self.current_controller, CellSelectorController):
        self.current_controller.view.pack_forget()

    # Создаем контроллер для доработки полученного файла tex.
    tex_content = self.current_controller.model.tex_content
    ipynb_images = self.current_controller.model.ipynb_images
    tex_parts = self.current_controller.model.tex_parts
    is_batch = self.current_controller.model.is_batch
    self.current_controller = FileFinalizationController(self, tex_content,
ipynb_images, tex_parts, is_batch)
    self.current_controller.view.pack()

def launch(self):
    self.show_main_menu() # Показать главное меню при запуске.
    self.root.mainloop()

```

1.2. Модели

Модели содержат бизнес-логику приложения и отвечают за обработку данных независимо от графического интерфейса.

Файл `src/MVC/models/main_menu_model.py` реализует загрузку одного notebook-файла или папки с notebook-файлами.

Файл `src/MVC/models/cell_selector_model.py` реализует подготовку выбранных ячеек, обработку Markdown-разметки, математических формул, изображений и экспорт выбранного содержимого в LaTeX.

Файл `src/MVC/models/file_finalization_model.py` реализует финальную обработку LaTeX-файла: добавление оглавления, нумерации заголовков, сохранение изображений, обработку удаленных изображений и пакетное сохранение в виде единого файла или модульной структуры.

1.2.1. Листинг 3. Файл `src/MVC/models/main_menu_model.py`

```

import nbformat
import os

from src.utils.formulab_exceptions import EmptyIpynbFileException

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата


```

class MainMenuModel:
    def __init__(self):
        self.file_path = None
        self.notebook_data = None
        self.notebook_sources = []

    def load_ipynb_file(self, file_path):
        try:
            with open(file_path, 'r', encoding='utf-8') as f:
                self.notebook_data = nbformat.read(f, as_version=4)
        except Exception as e:
            raise Exception(f"Ошибка загрузки файла: {e}")

        # Проверка на наличие ячеек в загруженном файле.
        if not self.notebook_data.get('cells'):
            raise EmptyIpyNbFileException()

        self.file_path = file_path
        self.notebook_sources = [{
            "name": os.path.basename(file_path),
            "path": file_path,
            "notebook": self.notebook_data,
        }]

    def load_notebooks_from_folder(self, folder_path):
        """Загружает все .ipynb файлы из папки и объединяет их в один notebook."""
        all_notebook_data = []
        self.notebook_sources = []
        for filename in sorted(os.listdir(folder_path), key=str.lower):
            if filename.endswith('.ipynb'):
                file_path = os.path.join(folder_path, filename)
                try:
                    with open(file_path, 'r', encoding='utf-8') as f:
                        notebook_data = nbformat.read(f, as_version=4)
                        all_notebook_data.append(notebook_data)
                        self.notebook_sources.append({

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

        "name": filename,
        "path": file_path,
        "notebook": notebook_data,
    })
except Exception as e:
    raise Exception(f"Ошибка при загрузке файла {filename}: {e}")

if not all_notebook_data:
    raise Exception("Нет файлов .ipynb в выбранной папке.")

self.notebook_data = self.__combine_notebooks(all_notebook_data)

# Проверка на наличие ячеек в загруженном файле.
if not self.notebook_data.get('cells'):
    raise EmptyIpyNbFileException(message="Папка содержит только пустые ipynb-
файлы. В них нет ни одной ячейки. "
                                         "Выберите папку, содержащую хотя бы
один непустой ipynb-файл")

self.file_path = folder_path

@staticmethod
def __combine_notebooks(notebook_data_list):
    """Объединяет ячейки всех ноутбуков в один."""
    combined_notebook = nbformat.v4.new_notebook()
    all_cells = []
    for notebook_data in notebook_data_list:
        all_cells.extend(notebook_data['cells'])
    combined_notebook.cells = all_cells
    return combined_notebook

```

1.2.2. Листинг 4. Файл src/MVC/models/cell_selector_model.py

```

import base64
import nbformat
import re
from copy import deepcopy
from nbconvert import LatexExporter
from nbconvert.filters import escape_latex

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```
from re import sub, DOTALL
```

```
from traitlets.config import Config
```

```
class CellSelectorModel:
```

```
    MARKDOWN_HEADING_STYLES = {
        1: ("\\LARGE", "1.2em", "0.8em"),
        2: ("\\Large", "1.1em", "0.7em"),
        3: ("\\large", "1em", "0.6em"),
        4: ("\\normalsize", "0.9em", "0.5em"),
        5: ("\\small", "0.8em", "0.45em"),
        6: ("\\footnotesize", "0.7em", "0.4em"),
    }
```

```
    def __init__(self, notebook_data, notebook_sources=None):
```

```
        self.notebook_data = notebook_data
        self.notebook_sources = notebook_sources or [{
            "name": "notebook.ipynb",
            "path": None,
            "notebook": notebook_data,
        }]
        self.tex_content = None
        self.tex_parts = []
        self.ipynb_images = {}
        self.is_batch = len(self.notebook_sources) > 1
```

```
    def get_cells(self, file_index=None):
```

```
        if file_index is None:
            return self.notebook_data['cells'] if self.notebook_data else []
        return self.notebook_sources[file_index]["notebook"].get('cells', [])
```

```
    def get_file_names(self):
```

```
        return [source["name"] for source in self.notebook_sources]
```

```
    def get_all_cell_indices(self, file_index=None):
```

```
        return list(range(1, len(self.get_cells(file_index)) + 1))
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

def get_markdown_cell_indices(self, file_index=None):
    return self.__get_cell_indices_by_predicate(
        lambda cell: cell.get('cell_type') == 'markdown',
        file_index
    )

def get_code_cell_indices(self, file_index=None):
    return self.__get_cell_indices_by_predicate(
        lambda cell: cell.get('cell_type') == 'code',
        file_index
    )

def get_output_cell_indices(self, file_index=None):
    return self.__get_cell_indices_by_predicate(
        lambda cell: bool(cell.get('outputs')),
        file_index
    )

def __get_cell_indices_by_predicate(self, predicate, file_index=None):
    return [
        cell_index
        for cell_index, cell in enumerate(self.get_cells(file_index), start=1)
        if predicate(cell)
    ]

def convert_to_tex(self, selected_indices):
    self.ipynb_images = {}
    self.tex_parts = []
    selected_cells = [deepcopy(self.notebook_data['cells'][i - 1]) for i in
selected_indices]
    tex_content = self.__convert_cells_to_tex(deepcopy(selected_cells))
    self.tex_parts.append({
        "name": self.notebook_sources[0]["name"],
        "tex_content": tex_content,
    })
    self.tex_content = tex_content

def convert_selection_to_tex(self, selected_cells_by_file):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

self.ipynb_images = {}
self.tex_parts = []
all_selected_cells = []

for file_index in sorted(selected_cells_by_file):
    selected_indices = sorted(selected_cells_by_file[file_index])
    if not selected_indices:
        continue

    source = self.notebook_sources[file_index]
    selected_cells = [
        deepcopy(source["notebook"]['cells'][i - 1])
        for i in selected_indices
    ]
    all_selected_cells.extend(deepcopy(selected_cells))
    self.tex_parts.append({
        "name": source["name"],
        "tex_content": self.__convert_cells_to_tex(deepcopy(selected_cells)),
    })

self.tex_content = self.__convert_cells_to_tex(all_selected_cells,
extract_images=False)

def __convert_cells_to_tex(self, selected_cells, extract_images=True):
    selected_cells = CellSelectorModel.__convert_markdown_headings(selected_cells)
    selected_cells =
CellSelectorModel.__clear_image_output_filenames(selected_cells)
    temp_notebook = nbformat.v4.new_notebook()
    temp_notebook.cells = selected_cells

    if extract_images:
        self.__extract_images(selected_cells)

    c = Config()
    c.ExtractOutputPreprocessor.output_filename_template = "image_{cell_index}
_{index}{extension}"

    latex_exporter = LatexExporter(config=c)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

try:
    tex_content, _ = latex_exporter.from_notebook_node(temp_notebook)
    return CellSelectorModel.__validate_tex(tex_content)
except Exception as e:
    raise Exception(f"Ошибка экспорта в LaTeX: {e}")

def __extract_images(self, cells):
    counter = len(self.ipynb_images) + 1
    for cell in cells:
        for output in cell.get('outputs', []):
            for mime, content in output.get('data', {}).items():
                if mime.startswith('image/'):
                    ext = mime.split('/')[1]
                    img_data = base64.b64decode(content)
                    name = f'image_{counter}.{ext}'
                    self.ipynb_images[name] = img_data
                    counter += 1

        for attachment in cell.get('attachments', {}).values():
            for mime, content in attachment.items():
                if mime.startswith('image/'):
                    ext = mime.split('/')[1]
                    img_data = base64.b64decode(content)
                    name = f'attach_{counter}.{ext}'
                    self.ipynb_images[name] = img_data
                    counter += 1

    @staticmethod
    def __clear_image_output_filenames(cells):
        for cell in cells:
            for output in cell.get('outputs', []):
                if not any(mime.startswith('image/') for mime in output.get('data', {})):
                    continue

            metadata = output.get('metadata')
            if not metadata:
                continue

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

        metadata.pop('filename', None)
        metadata.pop('filenames', None)

    return cells

@staticmethod
def __convert_markdown_headings(cells):
    for cell in cells:
        if cell.get('cell_type') != 'markdown':
            continue

        source = cell.get('source', '')
        if isinstance(source, list):
            source = ''.join(source)

        cell['source'] = CellSelectorModel.__replace_markdown_headings(source)

    return cells

@staticmethod
def __replace_markdown_headings(markdown_source):
    result_lines = []
    in_fenced_block = False
    fence_char = None
    fence_length = 0

    for line in markdown_source.splitlines(keepends=True):
        line_body = line.rstrip('\r\n')
        line_ending = line[len(line_body):]

        fence_match = re.match(r'^[ \t]{0,3}(\{3,}|~{3,})', line_body)
        if fence_match:
            marker = fence_match.group(1)
            marker_char = marker[0]
            marker_length = len(marker)
            if not in_fenced_block:
                in_fenced_block = True

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

        fence_char = marker_char
        fence_length = marker_length
    elif marker_char == fence_char and marker_length >= fence_length:
        in_fenced_block = False
        fence_char = None
        fence_length = 0
    result_lines.append(line)
    continue

if in_fenced_block:
    result_lines.append(line)
    continue

heading_match = re.match(r'^[ \t]{0,3}({1,6})(?!#)(?:[ \t]+|$)(.*)$',
line_body)

if not heading_match:
    result_lines.append(line)
    continue

level = len(heading_match.group(1))
heading_text = heading_match.group(2).strip()
heading_text = re.sub(r'[ \t]+#+[ \t]*$', '', heading_text).strip()
escaped_heading = escape_latex(heading_text)
size_command, top_space, bottom_space =
CellSelectorModel.MARKDOWN_HEADING_STYLES[level]

    result_lines.append(
        rf"\FormuLabHeading{{{level}}}{escaped_heading}" + "\n"
        rf"\par\addvspace{{{top_space}}}\noindent{{{size_command}}\bfseries
{escaped_heading}}}"
        rf"\par\nobreak\vspace{{{bottom_space}}}{line_ending}"
    )

return ''.join(result_lines)

@staticmethod
def __validate_tex(tex_content):
    tex_lines = tex_content.split("\n")

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата


```

tex_lines = CellSelectorModel.__add_required_libraries(tex_lines)
tex_lines = CellSelectorModel.__comment_title(tex_lines)
tex_lines = CellSelectorModel.__remove_labels(tex_lines)
tex_lines = CellSelectorModel.__handle_long_math_formulas(tex_lines)
return "\n".join(tex_lines)

@staticmethod
def __comment_title(tex_lines):
    return [
        "% \\maketitle" if line.strip() == "\\maketitle" else line
        for line in tex_lines
    ]

@staticmethod
def __add_required_libraries(tex_lines):
    header_insert = (
        "\\usepackage[T2A]{fontenc}\\n"
        "\\usepackage[utf8]{inputenc}\\n"
        "\\usepackage[russian]{babel}\\n"
        "\\usepackage{breqn}\\n"
        "\\usepackage{url}\\n"
        "\\newcommand{\\pandocbounded}[1]{\\n"
        "    \\adjustbox{max size={\\linewidth}{\\paperheight}}{#1}\\n"
        "}\\n"
    )

    if tex_lines and tex_lines[0].startswith("\\documentclass"):
        tex_lines.insert(1, header_insert)

    return tex_lines

@staticmethod
def __remove_labels(tex_lines):
    return [
        sub(r'\\label\\{.*?}', '', line)
        for line in tex_lines
    ]

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

@staticmethod
def __handle_long_math_formulas(tex_lines):
    threshold = 200
    tex_str = "\n".join(tex_lines)

    def replace_display_math(match):
        content = match.group(1).strip()
        if not content:
            return match.group(0)
        return "\\begin{dmath*}\n" + content + "\n\\end{dmath*}"

    tex_str = sub(r'\\\[s*(.*?)\s*\\\[', replace_display_math, tex_str,
flags=DOTALL)
    tex_str = sub(r'\$\$s*(.*?)\s*\$\$', replace_display_math, tex_str,
flags=DOTALL)

    def replace_inline_math(match):
        content = match.group(1).strip()
        if len(content) > threshold:
            return "\\begin{dmath*}\n" + content + "\n\\end{dmath*}"
        return match.group(0)

    tex_str = sub(r'\$\displaystyle\s*(.*?)\$', replace_inline_math, tex_str,
flags=DOTALL)
    return tex_str.split("\n")

```

1.2.3. Листинг 5. Файл src/MVC/models/file_finalization_model.py

```

import os
import re
import urllib.request
from tkinter import filedialog
from src.utils.formulab_exceptions import FileNotSelectedException

class FileFinalizationModel:
    FORMULAB_HEADING_PATTERN = re.compile(
        r'\\FormuLabHeading\{(?P<level>[1-6])\}\{(?P<title>(?:\\[{}]|[^{}])*)\}'
    )
    FORMULAB_TOC_LEVELS = {

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

1: "section",
2: "subsection",
3: "subsubsection",
4: "paragraph",
5: "subparagraph",
6: "subparagraph",
}

```

```

def __init__(self, intermediate_tex_content, ipynb_images, tex_parts=None,
is_batch=False):

```

```

    self.intermediate_tex_content = intermediate_tex_content
    self.ipynb_images = ipynb_images
    self.tex_parts = tex_parts or []
    self.final_tex_parts = []
    self.is_batch = is_batch
    self.final_tex_content = None

```

```

def refine_file(self, include_toc=False, include_headers_numeration=False):

```

```

    """Дорабатывает tex-файл на основе выбранных опций."""

```

```

    self.final_tex_content = self.intermediate_tex_content

```

```

    if include_toc:

```

```

        self.__add_table_of_contents()

```

```

    self.__apply_formulab_heading_options(

```

```

        include_toc=include_toc,

```

```

        include_headers_numeration=include_headers_numeration

```

```

    )

```

```

    if include_headers_numeration:

```

```

        self.__add_header_numeration()

```

Случай, при котором необходимо принудительно включить нумерованные заголовки в оглавление.

```

    if include_toc and not include_headers_numeration:

```

```

        self.__add_not_numbered_headers_to_toc()

```

```

    self.final_tex_parts = []

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

for part in self.tex_parts:
    part_model = FileFinalizationModel(part["tex_content"], {})
    part_model.refine_file(
        include_toc=include_toc,
        include_headers_numeration=include_headers_numeration
    )
    self.final_tex_parts.append({
        "name": part["name"],
        "tex_content": part_model.final_tex_content,
    })

def save_file(self, strategy="merge"):
    """Сохраняет доработанный tex-файл."""
    if self.final_tex_content is None:
        raise ValueError("Нет содержимого для сохранения.")

    if strategy == "modular" and self.is_batch:
        self.__save_modular_files()
        return

    file_path = filedialog.asksaveasfilename(defaultextension=".tex",
filetypes=[("LaTeX files", "*.tex")])

    if not file_path:
        raise FileNotSelectedException()

    # Определение папки для картинок рядом с tex-файлом.
    base, _ = os.path.splitext(os.path.basename(file_path))
    folder_name = f"{base}_FormuLab_images"
    img_folder = os.path.join(os.path.dirname(file_path), folder_name)
    remote_image_map = {}
    self.final_tex_content, _ = self.__localize_remote_image_references(
        self.final_tex_content,
        img_folder,
        folder_name,
        remote_image_map,
        0
    )

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

# Сохранение изображений в отдельной папке, если они есть.
if self.ipynb_images:
    # Шаг 1: сохранение изображений на ПК в отдельной папке.
    self.__save_images(img_folder)

    # Шаг 2: переименовывание файлов в latex-коде.
    self.__rename_files_in_latex_code()

    # Шаг 3: обновление путей в tex-контенте, добавляя папку перед названием
    файла.
    self.__add_image_folder_info(folder_name)

with open(file_path, "w", encoding="utf-8") as f:
    f.write(self.final_tex_content)

def __save_modular_files(self):
    output_dir = filedialog.askdirectory(title="Выберите папку для модульной
структуры")
    if not output_dir:
        raise FileNotSelectedException()

    parts_dir_name = "parts"
    parts_dir = os.path.join(output_dir, parts_dir_name)
    os.makedirs(parts_dir, exist_ok=True)

    folder_name = "main_FormuLab_images"
    saved_image_names = self.__get_saved_image_names()
    image_counter = 0
    if self.ipynb_images:
        self.__save_images(os.path.join(output_dir, folder_name))

    tex_parts = self.final_tex_parts or self.tex_parts
    used_part_names = set()
    input_paths = []
    remote_image_map = {}
    remote_image_counter = 0

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

for part_index, part in enumerate(tex_parts, start=1):
    part_filename = self.__unique_part_filename(part["name"], used_part_names,
part_index)
    part_path = os.path.join(parts_dir, part_filename)
    _, part_body = self.__split_latex_document(part["tex_content"])
    part_body = self.__remove_table_of_contents_commands(part_body)
    part_body, remote_image_counter = self.__localize_remote_image_references(
        part_body,
        os.path.join(output_dir, folder_name),
        folder_name,
        remote_image_map,
        remote_image_counter
    )

    if self.ipynb_images:
        part_body, image_counter = self.__rename_files_in_latex_content(
            part_body,
            {},
            saved_image_names,
            image_counter
        )
        part_body = self.__add_image_folder_info_to_content(
            part_body,
            folder_name,
            saved_image_names
        )

    with open(part_path, "w", encoding="utf-8") as f:
        f.write(part_body.strip() + "\n")

    input_paths.append(f"{parts_dir_name}/{part_filename}")

preamble, merged_body = self.__split_latex_document(self.final_tex_content)
main_lines = [
    preamble.rstrip(),
    "\\begin{document}",
]

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

if "\\tableofcontents" in merged_body:
    if "\\setcounter{tocdepth}{5}" in merged_body:
        main_lines.append("\\setcounter{tocdepth}{5}")
    main_lines.append("\\tableofcontents")

main_lines.extend(
    f"\\input{{{input_path}}}"
    for input_path in input_paths
)
main_lines.append("\\end{document}")

with open(os.path.join(output_dir, "main.tex"), "w", encoding="utf-8") as f:
    f.write("\n".join(line for line in main_lines if line) + "\n")

def __save_images(self, output_dir):
    """
    Сохраняет все извлеченные картинки в указанную папку.
    """
    if not self.ipynb_images:
        return
    os.makedirs(output_dir, exist_ok=True)
    for saved_name, data in zip(self.__get_saved_image_names(),
self.ipynb_images.values()):
        with open(os.path.join(output_dir, saved_name), 'wb') as f:
            f.write(data)

def __get_saved_image_names(self):
    saved_names = []
    for index, original_name in enumerate(self.ipynb_images, start=1):
        _, ext = os.path.splitext(original_name)
        saved_names.append(f"image_{index}{ext or '.png'}")
    return saved_names

def __rename_files_in_latex_code(self):
    r"""
    Переименовывает файлы в latex-коде.

```

Случай 1:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```
\\adjustimage{...}{old.png} ->
```

```
\\adjustimage{...}{image_1.png}
```

Случай 2:

```
\\pandocbounded{\\includegraphics[...] {old.png}} ->
```

```
\\pandocbounded{\\includegraphics[...] {image_1.png}}
```

```
"""
```

```
self.final_tex_content = self.__rename_files_in_latex_content(
    self.final_tex_content,
    {},
    self.__get_saved_image_names(),
    0
)[0]
```

```
@staticmethod
```

```
def __rename_files_in_latex_content(tex_content, image_name_map, saved_image_names,
start_index):
```

```
    next_image_index = start_index
```

```
def repl(match):
```

```
    nonlocal next_image_index
```

```
    if match.group(1):
```

```
        original_name = match.group(2)
```

```
        if not
```

```
FileFinalizationModel.__is_nbconvert_generated_image_name(original_name):
```

```
            return match.group(0)
```

```
            new_name, next_image_index =
```

```
FileFinalizationModel.__get_mapped_image_name(
```

```
            original_name,
```

```
            image_name_map,
```

```
            saved_image_names,
```

```
            next_image_index
```

```
        )
```

```
        return match.group(1) + "{" + new_name + "}"
```

```
    original_name = match.group(4)
```

```
    if not
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата


```

FileFinalizationModel.__is_nbconvert_generated_image_name(original_name):
    return match.group(0)
    new_name, next_image_index = FileFinalizationModel.__get_mapped_image_name(
        original_name,
        image_name_map,
        saved_image_names,
        next_image_index
    )
    return match.group(3) + new_name + "}"

image_pattern = re.compile(
    r'(\adjustimage\{(?:[^\}]*\}|{[^\}]*\})\{([^\}]+\}\}'
    r'|'
    r'(\pandocbounded\{\includegraphics\[^\]*\}\{([^\}]+\}\}'
)
return image_pattern.sub(repl, tex_content), next_image_index

@staticmethod
def __is_nbconvert_generated_image_name(image_name):
    normalized_image_name = image_name.replace("\\", "/").split("/")[-1]
    return bool(re.fullmatch(r'image_\d+_\d+\. [A-Za-z0-9+]',
normalized_image_name))

@staticmethod
def __get_mapped_image_name(original_name, image_name_map, saved_image_names,
next_image_index):
    normalized_original_name = original_name.replace("\\", "/").split("/")[-1]
    if normalized_original_name not in image_name_map:
        if next_image_index >= len(saved_image_names):
            raise ValueError(
                "Количество ссылок на изображения в LaTeX превышает количество "
                "извлеченных изображений из notebook."
            )
        image_name_map[normalized_original_name] =
saved_image_names[next_image_index]
        next_image_index += 1
    return image_name_map[normalized_original_name], next_image_index

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```
def __add_image_folder_info(self, folder_name):
```

```
    r"""
```

```
    Добавляет префикс папки к путям изображений в tex-контенте.
```

```
    Случай 1:
```

```
        \\adjustimage{...}{image_1.png} ->
```

```
        \\adjustimage{...}{folder_name/image_1.png}
```

```
    Случай 2:
```

```
        \\pandocbounded{\\includegraphics[...]{image_1.png}} ->
```

```
        \\pandocbounded{\\includegraphics[...]{folder_name/image_1.png}}
```

```
    """
```

```
    saved_image_names = self.__get_saved_image_names()
```

```
    # Случай 1: обработка adjustimage.
```

```
    pattern = r'\\adjustimage\\{(?:[^{}]|\\{[^{}]*\\})*\\}\\{([{}]+)\\}'
```

```
    self.final_tex_content = re.sub(
```

```
        pattern,
```

```
        lambda match: self.__add_folder_to_image_match(match, folder_name,
saved_image_names),
```

```
        self.final_tex_content
```

```
    )
```

```
    # Случай 2: обработка pandocbounded + includegraphics.
```

```
    pattern2 = r'\\pandocbounded\\{\\includegraphics\\{[^\\]*\\}\\{([{}]+)\\}'
```

```
    self.final_tex_content = re.sub(
```

```
        pattern2,
```

```
        lambda match: self.__add_folder_to_image_match(match, folder_name,
saved_image_names),
```

```
        self.final_tex_content
```

```
    )
```

```
@staticmethod
```

```
def __add_image_folder_info_to_content(tex_content, folder_name, saved_image_names):
```

```
    pattern = r'\\adjustimage\\{(?:[^{}]|\\{[^{}]*\\})*\\}\\{([{}]+)\\}'
```

```
    tex_content = re.sub(
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

    pattern,
    lambda match: FileFinalizationModel.__add_folder_to_image_match(
        match,
        folder_name,
        saved_image_names
    ),
    tex_content
)

pattern2 = r'(\pandocbounded\{\includegraphics\[^\]]*\}\{([^\}]+\})\}'
return re.sub(
    pattern2,
    lambda match: FileFinalizationModel.__add_folder_to_image_match(
        match,
        folder_name,
        saved_image_names
    ),
    tex_content
)

@staticmethod
def __add_folder_to_image_match(match, folder_name, saved_image_names):
    image_name = match.group(2)
    normalized_image_name = image_name.replace("\\", "/").split("/")[-1]
    if normalized_image_name not in saved_image_names:
        return match.group(0)
    return match.group(1) + "{" + folder_name + "/" + image_name + "}"

@staticmethod
def __localize_remote_image_references(
    tex_content,
    image_output_dir,
    latex_folder_name,
    remote_image_map,
    remote_image_counter
):
    def repl(match):
        nonlocal remote_image_counter

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

image_name = match.group("image_name")
if not FileFinalizationModel.__is_remote_image_reference(image_name):
    return match.group(0)

local_image_name = remote_image_map.get(image_name)
if not local_image_name:
    downloaded_image =
FileFinalizationModel.__download_remote_image(image_name)
    if not downloaded_image:
        return FileFinalizationModel.__remote_image_fallback(image_name)

    remote_image_counter += 1
    image_data, image_extension = downloaded_image
    local_image_name = f"remote_image_{remote_image_counter}
{image_extension}"
    os.makedirs(image_output_dir, exist_ok=True)
    with open(os.path.join(image_output_dir, local_image_name), "wb") as
image_file:
        image_file.write(image_data)
        remote_image_map[image_name] = local_image_name

    return match.group("prefix") + latex_folder_name + "/" + local_image_name +
match.group("suffix")

pandocbounded_pattern = re.compile(
    r'(?P<prefix>\\pandocbounded\\{\\includegraphics(?:\\[[^\\]]*\\)?\\}) '
    r'(?P<image_name>[^{}]+)'
    r'(?P<suffix>\\}) '
)
adjustimage_pattern = re.compile(
    r'(?P<prefix>\\adjustimage\\{(?:[^{}]|\\[[^{}]*\\])*\\}\\}) '
    r'(?P<image_name>[^{}]+)'
    r'(?P<suffix>\\}) '
)
includegraphics_pattern = re.compile(
    r'(?P<prefix>\\includegraphics(?:\\[[^\\]]*\\)?\\}) '
    r'(?P<image_name>[^{}]+)'
    r'(?P<suffix>\\}) '

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

)

```
tex_content = pandocbounded_pattern.sub(repl, tex_content)
tex_content = adjustimage_pattern.sub(repl, tex_content)
tex_content = includegraphics_pattern.sub(repl, tex_content)
return tex_content, remote_image_counter
```

```
@staticmethod
```

```
def __is_remote_image_reference(image_name):
    return image_name.startswith(("http://", "https://"))
```

```
@staticmethod
```

```
def __download_remote_image(image_url):
    try:
        request = urllib.request.Request(
            image_url,
            headers={"User-Agent": "Mozilla/5.0"}
        )
        with urllib.request.urlopen(request, timeout=20) as response:
            image_data = response.read()
            content_type = response.headers.get_content_type()
    except Exception:
        return None
```

```
    image_extension = FileFinalizationModel.__detect_image_extension(image_data,
content_type)
    if not image_extension:
        return None
    return image_data, image_extension
```

```
@staticmethod
```

```
def __detect_image_extension(image_data, content_type):
    if image_data.startswith(b"\x89PNG\r\n\x1a\n"):
        return ".png"
    if image_data.startswith(b"\xff\xd8\xff"):
        return ".jpg"
    if image_data.startswith(b"%PDF"):
        return ".pdf"
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

if image_data.startswith((b"GIF87a", b"GIF89a")):
    return ".gif"

content_type_extensions = {
    "image/png": ".png",
    "image/jpeg": ".jpg",
    "image/jpg": ".jpg",
    "application/pdf": ".pdf",
    "image/gif": ".gif",
}

return content_type_extensions.get(content_type)

@staticmethod
def __remote_image_fallback(image_url):
    return f"\\noindent Remote image: \\url{{{image_url}}}"

@staticmethod
def __split_latex_document(tex_content):
    begin_marker = "\\begin{document}"
    end_marker = "\\end{document}"
    begin_index = tex_content.find(begin_marker)

    if begin_index == -1:
        return "", tex_content

    preamble = tex_content[:begin_index]
    body_start = begin_index + len(begin_marker)
    end_index = tex_content.rfind(end_marker)
    if end_index == -1:
        return preamble, tex_content[body_start:]

    return preamble, tex_content[body_start:end_index]

@staticmethod
def __remove_table_of_contents_commands(tex_content):
    tex_content = re.sub(r'^[ \t]*\\setcounter{tocdepth}\\{5\\}[ \t]*\\r?\\n?', '',
tex_content, flags=re.MULTILINE)
    tex_content = re.sub(r'^[ \t]*\\tableofcontents[ \t]*\\r?\\n?', '', tex_content,

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

flags=re.MULTILINE)
    return tex_content

    @staticmethod
    def __unique_part_filename(source_name, used_names, part_index):
        base_name, _ = os.path.splitext(source_name)
        safe_name = re.sub(r'^A-Za-z0-9_-]+', '_', base_name).strip('_')
        if not safe_name:
            safe_name = f"notebook_{part_index}"

        candidate = f"{safe_name}.tex"
        suffix = 2
        while candidate.lower() in used_names:
            candidate = f"{safe_name}_{suffix}.tex"
            suffix += 1

        used_names.add(candidate.lower())
        return candidate

    def __add_table_of_contents(self):
        """Добавляет оглавление в tex-файл."""
        self.final_tex_content = self.final_tex_content.replace(
            "\\begin{document}",
            "\\begin{document}\n\\setcounter{tocdepth}{5}\n\\tableofcontents",
            1
        )

    def __apply_formulab_heading_options(self, include_toc=False,
include_headers_numeration=False):
        tex_lines = self.final_tex_content.splitlines(keepends=True)
        new_tex_lines = []
        heading_counters = [0] * 6
        pending_heading = None

        for line in tex_lines:
            line_without_markers = []
            current_position = 0
            for heading_match in self.FORMULAB_HEADING_PATTERN.finditer(line):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

line_without_markers.append(line[current_position:heading_match.start()])
    level = int(heading_match.group("level"))
    title = heading_match.group("title")
    heading_number = None

    if include_headers_numeration:
        heading_number = self.__next_heading_number(heading_counters, level)

    if include_toc:
        toc_title = f"{heading_number} {title}" if heading_number else title
        toc_level = self.FORMULAB_TOC_LEVELS[level]
        if line_without_markers and not
line_without_markers[-1].endswith(("\\n", "\\r")):
            line_without_markers.append("\\n")
            line_without_markers.append(
                f"\\phantomsection\\n\\addcontentsline{{toc}}{{{toc_level}}}{
{{{toc_title}}}}\\n"
            )

        pending_heading = {
            "title": title,
            "number": heading_number,
        }
        current_position = heading_match.end()

    if current_position:
        line_without_markers.append(line[current_position:])
        line = "".join(line_without_markers)

        if pending_heading and pending_heading["title"] in line and "\\
\\addcontentsline" not in line:
            if pending_heading["number"]:
                line = line.replace(
                    pending_heading["title"],
                    f"{pending_heading['number']} {pending_heading['title']}",
                    1
                )

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата


```

pending_heading = None

new_tex_lines.append(line)

self.final_tex_content = "".join(new_tex_lines)

@staticmethod
def __next_heading_number(heading_counters, level):
    level_index = level - 1
    for index in range(level_index):
        if heading_counters[index] == 0:
            heading_counters[index] = 1

    heading_counters[level_index] += 1

    for index in range(level, len(heading_counters)):
        heading_counters[index] = 0

    return ".".join(str(counter) for counter in heading_counters[:level] if counter)

def __add_header_numeration(self):
    """Добавляет нумерацию заголовков h1-h5 в tex-файл."""
    # Разбиваем исходный tex-файл построчно.
    tex_lines = self.final_tex_content.split("\n")

    # Убираем * из секционных команд для возврата нумерации заголовков h1-h5.
    tex_lines = [
        line if not any(
            line.strip().startswith(command) for command in [
                "\\section",
                "\\subsection",
                "\\subsubsection",
                "\\paragraph",
                "\\subparagraph",
            ]
        )
        else line.replace("\\section*", "\\section").replace("\\subsection*", "\\subsubsection").replace(
            "\\subsubsection*", "\\subsubsection").replace("\\paragraph*", "\\subparagraph")
    ]

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

\paragraph").replace(
    "\\subparagraph*", "\\subparagraph")
    for line in tex_lines
]

self.final_tex_content = "\n".join(tex_lines)

def __add_not_numbered_headers_to_toc(self):
    """Добавляет нумерованные заголовки в оглавление."""
    tex_lines = self.final_tex_content.split("\n")
    new_tex_lines = []

    for i, line in enumerate(tex_lines):
        # Проверяем, начинается ли строка с нумерованного заголовка.
        if line.strip().startswith((
            "\\section*",
            "\\subsection*",
            "\\subsubsection*",
            "\\paragraph*",
            "\\subparagraph*",
        )):
            # Определяем уровень заголовка.
            if line.strip().startswith("\\section*"):
                level = "section"
            elif line.strip().startswith("\\subsection*"):
                level = "subsection"
            elif line.strip().startswith("\\subsubsection*"):
                level = "subsubsection"
            elif line.strip().startswith("\\paragraph*"):
                level = "paragraph"
            else:
                level = "subparagraph"

            # Извлекаем название заголовка (может быть на одной или нескольких
строках).
            header_content = line.split("{", 1)[1] if "{" in line else ""
            if not header_content.endswith("}"):
                for j in range(i + 1, len(tex_lines)):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

        header_content += tex_lines[j].strip()
        if tex_lines[j].strip().endswith("{}"):
            break
    header_content = header_content.rstrip("{}")

    # Добавляем строку для оглавления ПЕРЕД самой строкой с заголовком.
    new_tex_lines.append(f"\\addcontentsline{{toc}}{{{level}}}{\\header_content}}")

    # Добавляем саму строку с заголовком.
    new_tex_lines.append(line)

self.final_tex_content = "\\n".join(new_tex_lines)

```

1.3. Представления

Представления содержат код графического интерфейса пользователя, реализованный средствами Tkinter.

Файл `src/MVC/views/main_menu_view.py` описывает главное окно приложения.

Файл `src/MVC/views/cell_selector_view.py` описывает окно выбора ячеек, список файлов при пакетной обработке, быстрые фильтры выбора и навигацию по страницам ячеек.

Файл `src/MVC/views/file_finalization_view.py` описывает окно предпросмотра и доработки итогового LaTeX-файла.

1.3.1. Листинг 6. Файл `src/MVC/views/main_menu_view.py`

```

import tkinter as tk

class MainMenuView(tk.Frame):
    def __init__(self, controller):
        super().__init__(controller.app.root)
        self.controller = controller
        self.show_screen()

    def show_screen(self):
        self.controller.app.root.title("FormuLab")

        tk.Label(self, text="Добро пожаловать в FormuLab!", font=("Arial",

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

14)).pack(pady=20)
    tk.Button(self, text="Загрузить ipynb-файл",
command=self.controller.load_file).pack(pady=10)
    tk.Button(self, text="Загрузить папку, содержащую ipynb-файлы",
command=self.controller.load_folder).pack(pady=10)
    tk.Button(self, text="Выход", command=self.quit).pack(pady=10)

```

1.3.2. Листинг 7. Файл src/MVC/views/cell_selector_view.py

```
import tkinter as tk
```

```
class CellSelectorView(tk.Frame):
```

```
    PAGE_SIZE = 250
```

```
    def __init__(self, controller, file_names):
```

```
        super().__init__(controller.app.root)
```

```
        self.controller = controller
```

```
        self.file_names = file_names
```

```
        self.current_file_index = 0
```

```
        self.current_page = 0
```

```
        self.cell_vars = {}
```

```
        self.canvas = None
```

```
        self.file_listbox = None
```

```
        self.show_screen()
```

```
    def show_screen(self):
```

```
        self.controller.app.root.title("FormuLab")
```

```
        self.pack(fill=tk.BOTH, expand=True)
```

```
        self.clear_frame()
```

```
        header = tk.Label(self, text="Выберите ячейки для конвертации", font=("Arial",
```

```
14))
```

```
        header.pack(pady=10, anchor="n")
```

```
        content_frame = tk.Frame(self)
```

```
        content_frame.pack(fill=tk.BOTH, expand=True)
```

```
        self.__show_file_list(content_frame)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

self.__show_current_file_cells(content_frame)
self.__show_action_buttons()

def __show_file_list(self, parent):
    file_frame = tk.Frame(parent, width=260)
    file_frame.pack(side=tk.LEFT, fill=tk.Y, padx=(8, 4), pady=4)
    file_frame.pack_propagate(False)

    tk.Label(file_frame, text="Файлы", font=("Arial", 11, "bold")).pack(anchor="w")
    self.file_listbox = tk.Listbox(file_frame, exportselection=False)
    self.file_listbox.pack(fill=tk.BOTH, expand=True, pady=(4, 0))
    self.file_listbox.bind("<<ListboxSelect>>", self.__on_file_selected)
    self.__refresh_file_list()

def __show_current_file_cells(self, parent):
    cells = self.controller.get_cells_for_file(self.current_file_index)
    total_pages = max(1, (len(cells) + self.PAGE_SIZE - 1) // self.PAGE_SIZE)
    self.current_page = min(self.current_page, total_pages - 1)

    cell_area = tk.Frame(parent)
    cell_area.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(4, 8), pady=4)

    tk.Label(
        cell_area,
        text=f"{self.file_names[self.current_file_index]} (стр. {self.current_page + 1}/{total_pages})",
        font=("Arial", 11, "bold")
    ).pack(anchor="w")

    list_frame = tk.Frame(cell_area)
    list_frame.pack(fill=tk.BOTH, expand=True)

    self.canvas = tk.Canvas(list_frame)
    self.canvas.bind_all("<MouseWheel>", self._on_canvas_mousewheel)
    self.canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

    scrollbar = tk.Scrollbar(list_frame, orient="vertical",
command=self.canvas.yview)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
self.canvas.config(yscrollcommand=scrollbar.set)

scrollable_frame = tk.Frame(self.canvas)
self.canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")

start = self.current_page * self.PAGE_SIZE
end = start + self.PAGE_SIZE
for offset, cell in enumerate(cells[start:end], start=start + 1):
    self.__show_cell(scrollable_frame, offset, cell)

scrollable_frame.update_idletasks()
self.canvas.config(scrollregion=self.canvas.bbox("all"))

def __show_cell(self, parent, cell_index, cell):
    cell_type = cell.get('cell_type', '')
    content = cell.get('source', '')
    cell_frame = tk.Frame(parent)
    cell_frame.pack(fill=tk.X, pady=5)

    text_frame = tk.Frame(cell_frame)
    text_frame.pack(fill=tk.X)

    text_widget = tk.Text(
        text_frame,
        height=5,
        width=80,
        wrap=tk.WORD,
        bg="white",
        fg="black",
        font=("Arial", 10)
    )
    text_widget.insert(tk.END, f"{cell_index}: [{cell_type}] {content}")
    text_widget.config(state=tk.DISABLED)
    text_widget.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    text_widget.bind("<MouseWheel>", self._on_text_mousewheel)

    scrollbar_inner = tk.Scrollbar(text_frame, orient="vertical",

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

command=text_widget.yview)
    scrollbar_inner.pack(side=tk.RIGHT, fill=tk.Y)
    text_widget.config(yscrollcommand=scrollbar_inner.set)

    tk.Checkbutton(
        cell_frame,
        text=f"Выбрать {cell_index}",
        variable=self.__get_cell_var(self.current_file_index, cell_index),
        command=self.__refresh_file_list
    ).pack(anchor="w", pady=2)

def __show_action_buttons(self):
    button_frame = tk.Frame(self)
    button_frame.pack(fill=tk.X, side=tk.BOTTOM)

    current_file_frame = tk.Frame(button_frame)
    current_file_frame.pack(fill=tk.X)
    tk.Button(current_file_frame, text="Выделить все (файл)", command=lambda:
self.controller.select_all_cells("current")).pack(side=tk.LEFT, padx=4, pady=3)
    tk.Button(current_file_frame, text="Снять выделение (файл)", command=lambda:
self.controller.deselect_all_cells("current")).pack(side=tk.LEFT, padx=4, pady=3)
    tk.Button(current_file_frame, text="Только текст (файл)", command=lambda:
self.controller.select_markdown_cells("current")).pack(side=tk.LEFT, padx=4, pady=3)
    tk.Button(current_file_frame, text="Только код (файл)", command=lambda:
self.controller.select_code_cells("current")).pack(side=tk.LEFT, padx=4, pady=3)
    tk.Button(current_file_frame, text="Только выходные данные (файл)",
command=lambda: self.controller.select_output_cells("current")).pack(side=tk.LEFT,
padx=4, pady=3)

    all_files_frame = tk.Frame(button_frame)
    all_files_frame.pack(fill=tk.X)
    tk.Button(all_files_frame, text="Выделить все (все)", command=lambda:
self.controller.select_all_cells("all")).pack(side=tk.LEFT, padx=4, pady=3)
    tk.Button(all_files_frame, text="Снять выделение (все)", command=lambda:
self.controller.deselect_all_cells("all")).pack(side=tk.LEFT, padx=4, pady=3)
    tk.Button(all_files_frame, text="Только текст (все)", command=lambda:
self.controller.select_markdown_cells("all")).pack(side=tk.LEFT, padx=4, pady=3)
    tk.Button(all_files_frame, text="Только код (все)", command=lambda:

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

self.controller.select_code_cells("all")).pack(side=tk.LEFT, padx=4, pady=3)
    tk.Button(all_files_frame, text="Только выходные данные (все)", command=lambda:
self.controller.select_output_cells("all")).pack(side=tk.LEFT, padx=4, pady=3)

    navigation_frame = tk.Frame(button_frame)
    navigation_frame.pack(fill=tk.X)

    cells = self.controller.get_cells_for_file(self.current_file_index)
    if self.current_page > 0:
        tk.Button(navigation_frame, text="Назад",
command=self.previous_page).pack(side=tk.LEFT, padx=4, pady=5)
        if (self.current_page + 1) * self.PAGE_SIZE < len(cells):
            tk.Button(navigation_frame, text="Далее",
command=self.next_page).pack(side=tk.LEFT, padx=4, pady=5)

        tk.Button(navigation_frame, text="Конвертировать",
command=self.controller.convert).pack(side=tk.LEFT, padx=4, pady=5)
        tk.Button(navigation_frame, text="Отмена",
command=self.controller.back).pack(side=tk.LEFT, padx=4, pady=5)

def __on_file_selected(self, _event):
    selection = self.file_listbox.curselection()
    if not selection:
        return
    self.current_file_index = selection[0]
    self.current_page = 0
    self.show_screen()

def __refresh_file_list(self):
    if not self.file_listbox:
        return
    self.file_listbox.delete(0, tk.END)
    for file_index, file_name in enumerate(self.file_names):
        marker = "✓" if self.__file_has_selected_cells(file_index) else " "
        self.file_listbox.insert(tk.END, f"{marker} {file_name}")
    self.file_listbox.selection_set(self.current_file_index)

def __file_has_selected_cells(self, file_index):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата


```

return any(
    var.get()
    for (selected_file_index, _cell_index), var in self.cell_vars.items()
    if selected_file_index == file_index
)

def clear_frame(self):
    for widget in self.winfo_children():
        widget.destroy()

def __get_cell_var(self, file_index, cell_index):
    key = (file_index, cell_index)
    if key not in self.cell_vars:
        self.cell_vars[key] = tk.BooleanVar()
    return self.cell_vars[key]

def set_selected_indices(self, file_index, selected_indices):
    selected_indices = set(selected_indices)
    cells = self.controller.get_cells_for_file(file_index)
    for cell_index in range(1, len(cells) + 1):
        self.__get_cell_var(file_index, cell_index).set(cell_index in
selected_indices)
    self.__refresh_file_list()

def set_selected_indices_by_file(self, selected_indices_by_file):
    for file_index, selected_indices in selected_indices_by_file.items():
        self.set_selected_indices(file_index, selected_indices)

def get_selected_cells_by_file(self):
    selected = {file_index: [] for file_index in range(len(self.file_names))}
    for (file_index, cell_index), var in self.cell_vars.items():
        if var.get():
            selected[file_index].append(cell_index)
    return {
        file_index: sorted(indices)
        for file_index, indices in selected.items()
    }

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

def get_selected_indices(self):
    if len(self.file_names) != 1:
        return []
    return self.get_selected_cells_by_file().get(0, [])

def next_page(self):
    self.current_page += 1
    self.show_screen()

def previous_page(self):
    self.current_page -= 1
    self.show_screen()

def _on_text_mousewheel(self, event):
    event.widget.yview_scroll(int(-1 * (event.delta / 120)), "units")
    return "break"

def _on_canvas_mousewheel(self, event):
    if self.canvas:
        self.canvas.yview_scroll(int(-1 * (event.delta / 120)), "units")

```

1.3.3. Листинг 8. Файл src/MVC/views/file_finalization_view.py

```

import tkinter as tk
from tkinter import scrolledtext

class FileFinalizationView(tk.Frame):
    def __init__(self, controller, tex_content, is_batch=False):
        super().__init__(controller.app.root)
        self.controller = controller
        self.tex_content = tex_content
        self.is_batch = is_batch
        self.is_table_of_contents_included = tk.BooleanVar()
        self.is_headers_numeration_included = tk.BooleanVar()
        self.export_strategy = tk.StringVar(value="merge")
        self.show_screen()

    def show_screen(self):

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

self.controller.app.root.title("FormuLab")

tk.Label(self, text="Доработка файла", font=("Arial", 14)).pack(pady=20)
tk.Checkbutton(self, text="Добавить оглавление",
variable=self.is_table_of_contents_included).pack(anchor="w")
tk.Checkbutton(self, text="Добавить нумерацию заголовков",
variable=self.is_headers_numeration_included).pack(anchor="w")

if self.is_batch:
    strategy_frame = tk.LabelFrame(self, text="Стратегия пакетной обработки")
    strategy_frame.pack(fill=tk.X, padx=10, pady=8)
    tk.Radiobutton(
        strategy_frame,
        text="Слияние в единый .tex файл",
        variable=self.export_strategy,
        value="merge"
    ).pack(anchor="w")
    tk.Radiobutton(
        strategy_frame,
        text="Модульная структура: main.tex + отдельные части",
        variable=self.export_strategy,
        value="modular"
    ).pack(anchor="w")

tk.Label(self, text="Предпросмотр .tex файла:").pack(pady=10)
preview = scrolledtext.ScrolledText(self, height=15, wrap=tk.WORD,
font=("Courier", 10))
preview.insert(tk.END, self.tex_content)
preview.config(state=tk.DISABLED)
preview.pack(expand=True, fill="both", padx=10, pady=10)

tk.Button(self, text="Сохранить файл",
command=self.controller.finalize_file).pack(pady=10)
tk.Button(self, text="Назад", command=self.controller.back).pack(pady=10)

```

1.4. Контроллеры

Контроллеры связывают графический интерфейс с моделями и обрабатывают действия оператора.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Файл `src/MVC/controllers/main_menu_controller.py` обрабатывает команды загрузки файла и загрузки папки.

Файл `src/MVC/controllers/cell_selector_controller.py` обрабатывает выбор ячеек, применение быстрых фильтров и запуск конвертации.

Файл `src/MVC/controllers/file_finalization_controller.py` обрабатывает команды финализации и сохранения итогового файла.

1.4.1. Листинг 9. Файл `src/MVC/controllers/main_menu_controller.py`

```
from tkinter import filedialog, messagebox

from src.MVC.models.main_menu_model import MainMenuModel
from src.MVC.views.main_menu_view import MainMenuView
from src.utils.formulab_exceptions import EmptyIpynbFileException

class MainMenuController:
    def __init__(self, app):
        self.app = app
        self.model = MainMenuModel()
        self.view = MainMenuView(self)

    def load_file(self):
        file_path = filedialog.askopenfilename(
            filetypes=[("Jupyter Notebook Files", "*.ipynb"), ("All Files", "*.*")],
            title="Выберите .ipynb-файл"
        )

        if file_path:
            try:
                self.model.load_ipynb_file(file_path)
                self.app.show_cell_selector()
            except EmptyIpynbFileException as e:
                messagebox.showerror("Ошибка", str(e))
            except Exception as e:
                messagebox.showerror("Ошибка", str(e))

    def load_folder(self):
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

folder_path = filedialog.askdirectory(
    title="Выберите папку с .ipynb-файлами",
    mustexist=True
)

if folder_path:
    try:
        self.model.load_notebooks_from_folder(folder_path)
        self.app.show_cell_selector()
    except EmptyIpyNbFileException as e:
        messagebox.showerror("Ошибка", str(e))
    except Exception as e:
        messagebox.showerror("Ошибка", str(e))

```

1.4.2. Листинг 10. Файл src/MVC/controllers/cell_selector_controller.py

```

from tkinter import messagebox

from src.MVC.models.cell_selector_model import CellSelectorModel
from src.MVC.views.cell_selector_view import CellSelectorView

class CellSelectorController:
    def __init__(self, app, notebook_data, notebook_sources=None):
        self.app = app
        self.model = CellSelectorModel(notebook_data, notebook_sources)
        self.view = CellSelectorView(self, self.model.get_file_names())

    def get_cells_for_file(self, file_index):
        return self.model.get_cells(file_index)

    def convert(self):
        selected_cells_by_file = self.view.get_selected_cells_by_file()
        if not any(selected_cells_by_file.values()):
            messagebox.showwarning("Предупреждение", "Выберите хотя бы одну ячейку!")
            return

        try:
            self.model.convert_selection_to_tex(selected_cells_by_file)

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```

        self.app.show_file_finalization()
    except Exception as e:
        messagebox.showerror("Ошибка", str(e))

def back(self):
    self.app.show_main_menu()

def select_all_cells(self, scope="current"):
    self.__apply_selection(scope, self.model.get_all_cell_indices)

def deselect_all_cells(self, scope="current"):
    if scope == "all":
        self.view.set_selected_indices_by_file({
            file_index: []
            for file_index in range(len(self.model.get_file_names()))
        })
    else:
        self.view.set_selected_indices(self.view.current_file_index, [])

def select_markdown_cells(self, scope="current"):
    self.__apply_selection(scope, self.model.get_markdown_cell_indices)

def select_code_cells(self, scope="current"):
    self.__apply_selection(scope, self.model.get_code_cell_indices)

def select_output_cells(self, scope="current"):
    self.__apply_selection(scope, self.model.get_output_cell_indices)

def __apply_selection(self, scope, index_getter):
    if scope == "all":
        self.view.set_selected_indices_by_file({
            file_index: index_getter(file_index)
            for file_index in range(len(self.model.get_file_names()))
        })
    else:
        file_index = self.view.current_file_index
        self.view.set_selected_indices(file_index, index_getter(file_index))

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1.4.3. Листинг 11. Файл src/MVC/controllers/file_finalization_controller.py

```

from tkinter import messagebox

from src.MVC.models.file_finalization_model import FileFinalizationModel
from src.MVC.views.file_finalization_view import FileFinalizationView
from src.utils.formulab_exceptions import FileNotSelectedException

class FileFinalizationController:
    def __init__(self, app, intermediate_tex_content, ipynb_images, tex_parts=None,
is_batch=False):
        self.app = app
        self.model = FileFinalizationModel(intermediate_tex_content, ipynb_images,
tex_parts, is_batch)
        self.view = FileFinalizationView(self, intermediate_tex_content, is_batch)

    def finalize_file(self):
        self.model.refine_file(
            include_toc=self.view.is_table_of_contents_included.get(),
            include_headers_numeration=self.view.is_headers_numeration_included.get()
        )

        try:
            self.model.save_file(self.view.export_strategy.get())
        except ValueError as e:
            messagebox.showerror("Ошибка", f"Не удалось сохранить файл: {e}")
            return
        except FileNotSelectedException:
            messagebox.showwarning("Предупреждение", "Вы не выбрали место для
сохранения.")
            return
        except Exception as e:
            messagebox.showerror("Неизвестная ошибка", f"Произошла ошибка при сохранении
файла: {e}")
            return

        self.app.show_main_menu()

```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

```
def back(self):
    self.app.show_cell_selector()
```

1.5. Утилиты

Утилиты содержат вспомогательные компоненты, используемые различными частями приложения.

Файл `src/utils/formulab_exceptions.py` содержит специализированные исключения приложения.

Файл `src/utils/windows_subprocess.py` содержит вспомогательные средства для скрытого запуска дочерних процессов Windows, используемых при вызове внешних инструментов, например Pandoc.

1.5.1. Листинг 12. Файл `src/utils/formulab_exceptions.py`

```
class FileNotSelectedException(Exception):
    """Исключение, выбрасываемое, когда файл не был выбран пользователем.

    Атрибуты:
        message (str): Сообщение об ошибке.
        dialog (str): Строка, описывающая, в каком контексте возникла ошибка.
    """

    def __init__(self, message="Файл не выбран", dialog="Диалог сохранения файла не был
завершён пользователем"):
        self.message = message
        self.dialog = dialog
        super().__init__(self.message)

    def __str__(self):
        return f"{self.message} - {self.dialog}"

class EmptyIpynbFileException(Exception):
    """Исключение, выбрасываемое, когда загружается пустой Jupyter Notebook без
ячеек."""

    def __init__(self, message="Файл пустой. В нем нет ни одной ячейки. Выберите файл,
содержащий хотя бы одну ячейку"):
        self.message = message
        super().__init__(self.message)
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1.5.2. Листинг 13. Файл src/utils/windows_subprocess.py

```

from subprocess import Popen as _Popen, CREATE_NO_WINDOW, STARTUPINFO,
STARTF_USESHOWWINDOW

# Monkey-патч для subprocess.Popen, чтобы скрыть окна подпроцессов Windows.
# Это нужно, чтобы при вызове внешних консольных программ (например, pandoc)
# не появлялись мерцающие окна терминала.
def popen_hidden(*args, **kwargs):
    # noinspection GrazieInspection
    """
    Обёртка для subprocess.Popen, позволяющая скрыть консольное окно
    в дочерних процессах на Windows.

    :param args: Позиционные аргументы, передаваемые оригинальному Popen
    :param kwargs: Именованные аргументы; сюда будет добавлен 'creationflags' и
    'startupinfo'
    :return: Экземпляр Popen, запущенный с настройками скрытия окна
    """
    # Устанавливаем флаг CREATE_NO_WINDOW, если он ещё не задан.
    # Это предотвращает появление консольного окна.
    kwargs.setdefault("creationflags", CREATE_NO_WINDOW)

    # Инициализируем или получаем существующий startupinfo для процесса.
    si = kwargs.get("startupinfo", STARTUPINFO())
    # Добавляем флаг SHOWWINDOW в startupinfo, чтобы окно было скрыто.
    si.dwFlags |= STARTF_USESHOWWINDOW
    # Сохраняем обновлённый объект startupinfo в аргументы.
    kwargs["startupinfo"] = si

    # Вызываем оригинальный Popen с нашими модификациями.
    return _Popen(*args, **kwargs)

```

Файлы `__init__.py` используются для обозначения каталогов как Python-пакетов. Так как данные файлы не содержат исполняемого кода, их листинги не приводятся.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.404-79 Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.401-78 Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79 Руководство оператора. Требования к содержанию и оформлению // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.301-79 Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. ГОСТ 7.32-2017 Отчет о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017.
15. Python.org официальная страница языка программирования Python [Электронный ресурс]. // URL: <https://www.python.org/> (Дата обращения: 02.12.2025, режим доступа: свободный)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

16. Tkinter Documentation официальная документация библиотеки Tkinter [Электронный ресурс]. // URL: <https://docs.python.org/3/library/tkinter.html> (Дата обращения: 03.12.2025, режим доступа: свободный)
17. Nbconvert Documentation официальная документация библиотеки Nbconvert [Электронный ресурс]. // URL: <https://nbconvert.readthedocs.io/en/stable/> (Дата обращения: 06.12.2025, режим доступа: свободный)
18. Nbformat Documentation - официальная документация библиотеки Nbformat [Электронный ресурс]. // URL: <https://nbformat.readthedocs.io/en/stable/> (Дата обращения: 09.12.2025, режим доступа: свободный)
19. Mastering Python Design Patterns паттерны проектирования на Python [Электронный ресурс]. // URL: <https://www.programmer-books.com/wp-content/uploads/2019/12/Mastering-Python-Design-Patterns.pdf> (Дата обращения: 23.12.2025, режим доступа: свободный)
20. Хорасанджян Л.А. ТЗ_Хорасанджян. Техническое задание к выпускной квалификационной работе по теме: «Desktop-приложение для обработки текста, полученного nbconvert и содержащего формулы Latex». — М.: НИУ ВШЭ, 2025.
21. Горячая И. В. Пользовательский интерфейс: назначение, принципы разработки и практические рекомендации [Электронный ресурс]. // URL: <https://al.cs.msu.ru/files/goryachaya.user.interface.2010.pdf> (Дата обращения: 01.12.2025, режим доступа: свободный)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.06.11-01 12 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]